Universidad del Valle de Guatemala
Facultad de Ingeniería — Ciencias de la Computación
Ciclo 1 — 2026

Proyecto 1 — Chat Multithread

Parte 1: Definición del Protocolo de Comunicación

CC3064 Sistemas Operativos

José Antonio Mérida Castejón
Javier Eduardo España Pacheco
Angel Esteban Esquit Hernández

27 de marzo de 2026

Índice

1. Introducción **2**

2. Arquitectura del Sistema **2**
2.1. Red 2
2.2. Comunicación 2
2.3. Concurrencia 2
2.4. Ejecución 3

3. Diseño del Protocolo de Mensajes **3**
3.1. Formato 3
3.2. Mensajes del Cliente al Servidor 3
3.3. Mensajes del Servidor al Cliente 4

4. Flujos de Comunicación **4**
4.1. Registro de Usuario 4
4.2. Broadcast (Chat General) 5
4.3. Mensaje Directo 5
4.4. Listado de Usuarios 6
4.5. Información de Usuario 6
4.6. Cambio de Estado 6
4.7. Inactividad 6
4.8. Desconexión 7

5. Códigos de Error **7**

6. Sincronización **7**
6.1. Read-Write Lock para la Lista de Usuarios 7
6.2. Mutex por Socket 8

1. Introducción

Este documento define el protocolo de comunicación para el chat multithread del Proyecto 1. Se describe cómo se conectan las máquinas, cómo se estructuran los mensajes entre cliente y servidor, qué hace el servidor con cada tipo de mensaje, y cómo se manejan los accesos concurrentes a los datos compartidos.

2. Arquitectura del Sistema

2.1. Red

El sistema funciona en una red local: varias computadoras conectadas a un mismo switch con cables Ethernet, todas en la misma subred (por ejemplo, `192.168.1.0/24`). Una máquina corre el servidor; las demás corren clientes.

Para configurar la red en cada máquina se usa `ip` de Linux:

```
ip address show                          # ver IPs actuales
sudo ip address add 192.168.1.10/24 dev eth0  # asignar IP a la interfaz
sudo ip link set eth0 up                 # encender la interfaz
ping 192.168.1.20                        # verificar conexion con otra
   maquina
```

Cada máquina necesita una IP distinta dentro de la misma subred. El `/24` indica que el prefijo de red son los primeros 3 octetos (o sea, todas las máquinas comparten `192.168.1.x`). La interfaz (`eth0` o similar) es el puerto Ethernet de la máquina conectado al switch.

2.2. Comunicación

Se usa TCP (`SOCK_STREAM`) porque garantiza que los mensajes lleguen completos y en orden, que es lo que necesitamos para un chat. El protocolo es texto plano sobre TCP, no se usa HTTP ni HTTPS ya que esos protocolos agregan headers y estructura que no necesitamos.

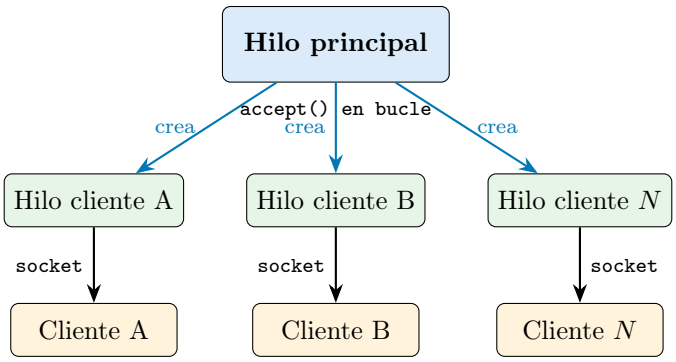
2.3. Concurrencia

El servidor sigue un modelo de un hilo por cliente. El hilo principal solo acepta conexiones nuevas. Cada conexión se delega a un hilo dedicado que maneja toda la comunicación con ese cliente hasta que se desconecta.

Se eligió este modelo porque es directo, cada hilo tiene un ciclo de recibir–procesar–responder sobre un solo socket.

Todos los hilos comparten una lista de usuarios conectados. Para protegerla se usa un **read-write lock** (`pthread_rwlock_t`) en vez de un mutex normal. La diferencia es que un mutex bloquea la lista para cualquier acceso; si un hilo está leyendo, nadie más puede leer. El rwlock permite que múltiples hilos lean la lista al mismo tiempo (por ejemplo, varios broadcasts en paralelo), y solo bloquea a todos cuando un hilo necesita modificarla (registro, desconexión, cambio de estado). Como la mayoría de operaciones son de lectura, esto evita cuellos de botella innecesarios.

Además, cada socket de cliente tiene un **mutex propio** (`pthread_mutex_t`) para evitar que dos hilos escriban al mismo socket a la vez (por ejemplo, dos broadcasts simultáneos intentando entregar mensajes al mismo destinatario).



2.4. Ejecución

El servidor recibe el puerto como argumento:

```
./chat_server <puerto>
```

El cliente envía su nombre de usuario, la IP del servidor y el puerto:

```
./chat_client <usuario> <IP_servidor> <puerto>
```

Por ejemplo, si el servidor corre en `192.168.1.10` puerto `8080`:

```
./chat_client alice 192.168.1.10 8080
```

3. Diseño del Protocolo de Mensajes

3.1. Formato

Texto plano, campos separados por `'|'`, terminados con `'\n'`:

```
TIPO|CAMPO1|CAMPO2|...|CAMPON\n
```

- **Texto plano:** se puede depurar con `netcat` directamente y parsear con `strtok()` en C.
- **Delimitador `'|'`:** poco frecuente en texto normal. Si el usuario lo escribe, se reemplaza por `'::'`.
- **Terminador `'\n'`:** permite leer línea por línea.
- **Máximo 4096 bytes:** suficiente para chat; permite buffers estáticos.

3.2. Mensajes del Cliente al Servidor

El cliente arma un string con el tipo de operación y los campos necesarios. El servidor separa por `'|'` y despacha según el primer campo.

Tipo	Campos	Descripción
`REGISTER`	usuario	Solicita registro con el nombre indicado.
`MSG_BROADCAST`	remitente, mensaje	Mensaje al chat general.
`MSG_DIRECT`	remitente, destino, mensaje	Mensaje privado a un usuario.

Tipo	Campos	Descripción
`LIST_USERS`	remitente	Pide la lista de usuarios conectados.
`GET_USER_INFO`	remitente, usuario	Pide la IP y estado de un usuario.
`CHANGE_STATUS`	remitente, estado	Cambia a ACTIVO, OCUPADO o INACTIVO.
`DISCONNECT`	remitente	Cierre voluntario de sesión.

3.3. Mensajes del Servidor al Cliente

El servidor responde al cliente que hizo la solicitud y, cuando corresponde, notifica a los demás.

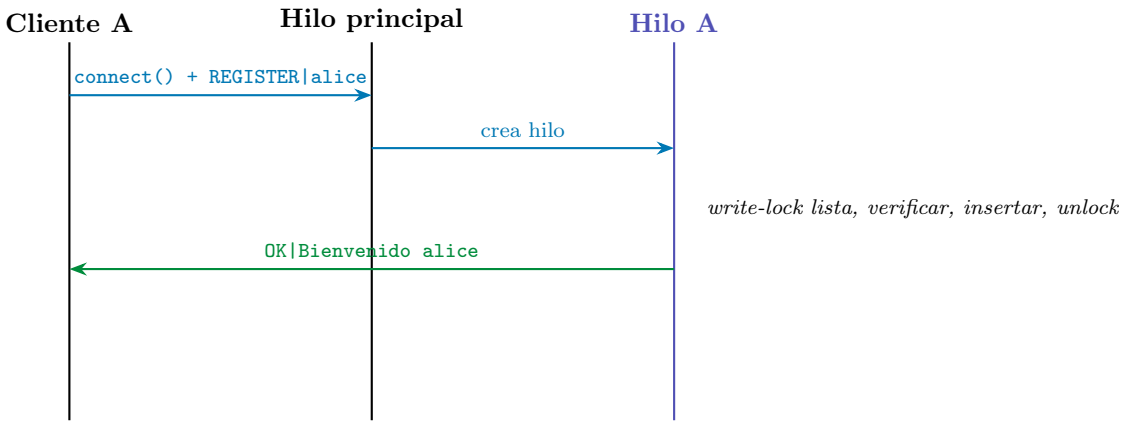
Tipo	Campos	Descripción
`OK`	mensaje	Confirmación de operación exitosa.
`ERROR`	código, mensaje	Respuesta de error con código numérico.
`SERVER_BROADCAST`	remitente, mensaje	Broadcast reenviado a todos.
`SERVER_DIRECT`	remitente, mensaje	Mensaje privado entregado.
`USER_LIST`	n, usuario:estado, ...	Lista de usuarios con estados.
`USER_INFO`	usuario, ip, estado	Info de un usuario específico.
`STATUS_UPDATE`	usuario, estado	Confirmación de cambio de estado.
`FORCED_STATUS`	usuario, INACTIVO	Estado forzado por inactividad.
`USER_JOINED`	usuario, ip	Un usuario se conectó.
`USER_LEFT`	usuario	Un usuario se desconectó.

4. Flujos de Comunicación

Los diagramas muestran cómo se mueven los mensajes entre el cliente, su hilo dedicado en el servidor, y la lista compartida de usuarios. Las líneas punteadas son accesos a la lista (protegidos por lock).

4.1. Registro de Usuario

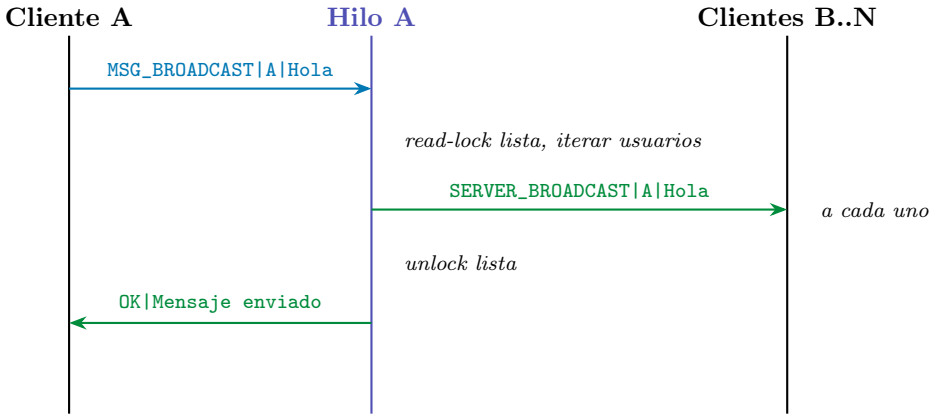
El cliente se conecta al servidor y envía su nombre de usuario. El hilo principal acepta la conexión, crea un hilo dedicado, y ese hilo ejecuta todo el registro. Toma el write-lock de la lista, verifica que el nombre y la IP no estén repetidos, inserta al usuario, suelta el lock, y responde. Desde la perspectiva del cliente es una sola operación (conectar y registrarse).



Errores posibles: `101` (nombre duplicado), `102` (IP duplicada), `103` (nombre inválido).

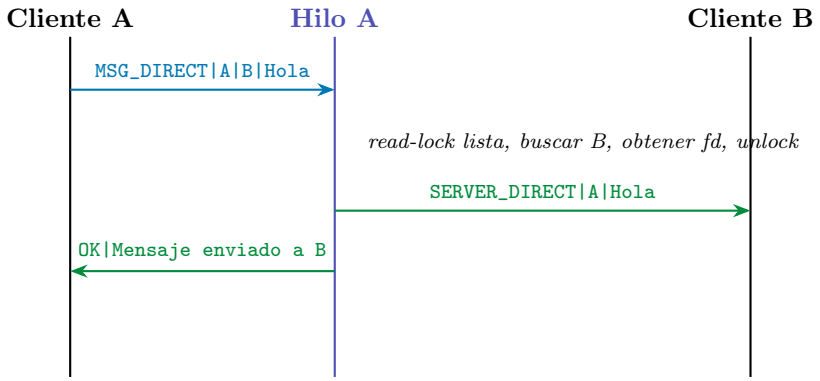
4.2. Broadcast (Chat General)

El hilo del cliente emisor recibe el mensaje, toma el read-lock de la lista, recorre todos los usuarios conectados y escribe el broadcast al socket de cada uno (tomando el lock del socket de cada destinatario antes de escribir). Al terminar, suelta el lock de la lista y responde al emisor.



4.3. Mensaje Directo

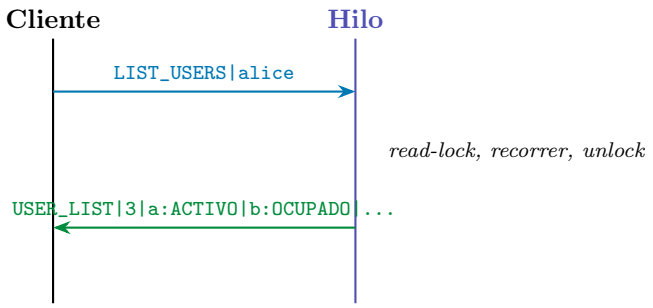
El hilo del emisor toma el read-lock de la lista, busca al destinatario, obtiene su socket, suelta el lock, y escribe directamente al socket del destinatario (con el lock de ese socket).



Errores posibles: `201` (usuario destino no existe).

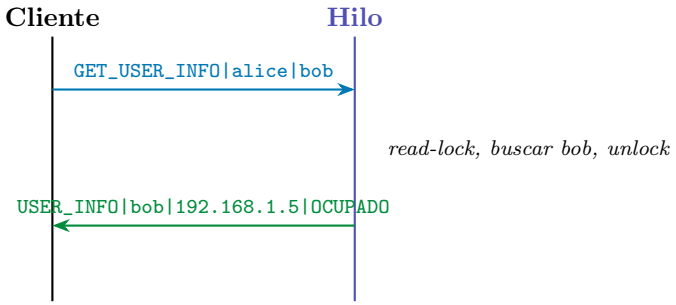
4.4. Listado de Usuarios

El hilo toma el read-lock, recorre la lista armando la respuesta con cada nombre y estado, suelta el lock, y envía.



4.5. Información de Usuario

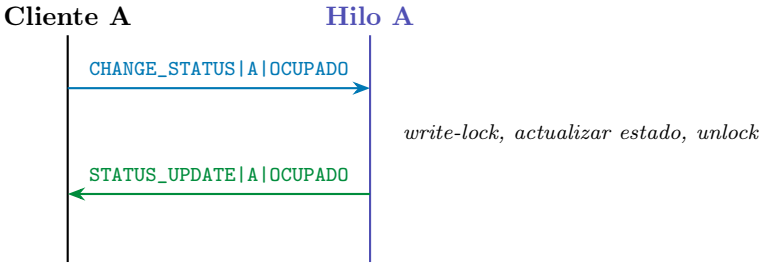
Mismo patrón: read-lock, buscar el usuario, obtener su IP y estado, unlock, responder.



Errores posibles: `201` (usuario no encontrado).

4.6. Cambio de Estado

El hilo toma el write-lock (porque modifica datos), actualiza el estado, suelta el lock, y responde al cliente.



Errores posibles: `301` (estado no válido).

4.7. Inactividad

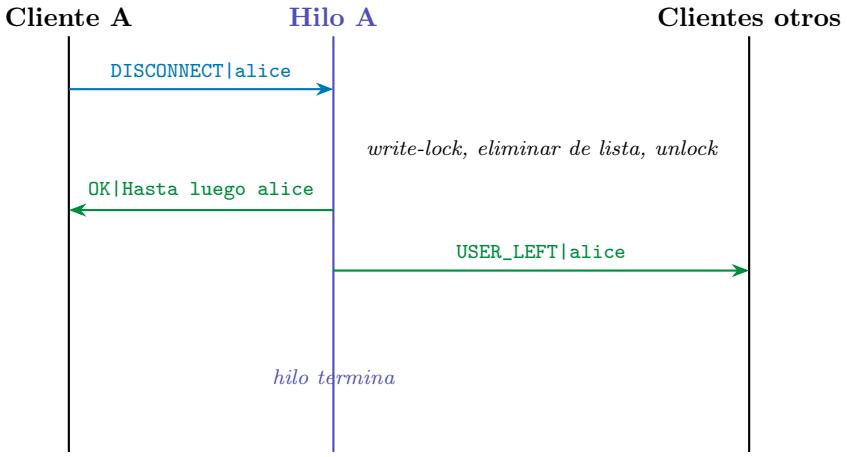
La inactividad se maneja **del lado del servidor**. Cada hilo de cliente lleva registro de la última vez que su cliente envió un mensaje. Si pasa el tiempo definido por `INACTIVITY_TIMEOUT` sin recibir

nada, el hilo cambia el estado del cliente a `INACTIVO` (tomando el write-lock) y le envía:

```
FORCED_STATUS|<usuario>|INACTIVO
```

4.8. Desconexión

El hilo recibe el `DISCONNECT`, toma el write-lock para eliminar al usuario de la lista, suelta el lock, responde al cliente, notifica a los demás, y termina.



Si la desconexión es abrupta (el socket se cierra sin `DISCONNECT`), el hilo lo detecta cuando `recv()` retorna 0 y ejecuta la misma limpieza.

5. Códigos de Error

Los códigos están agrupados por centenas según la operación.

Código	Contexto	Significado
101	`REGISTER`	Nombre de usuario ya registrado.
102	`REGISTER`	La IP de origen ya tiene una sesión activa.
103	`REGISTER`	Nombre inválido (vacío o contiene `'|'`).
201	`MSG_DIRECT / GET_USER_INFO`	Usuario destino no existe o no está conectado.
301	`CHANGE_STATUS`	Estado solicitado no es válido.
401	`General`	Mensaje con formato desconocido o malformado.
501	`MSG_BROADCAST / MSG_DIRECT`	Cliente tiene estado INACTIVO; debe cambiar de estado.

6. Sincronización

6.1. Read-Write Lock para la Lista de Usuarios

La lista de usuarios es el recurso compartido principal del servidor. En un chat, la gran mayoría de operaciones son de lectura: broadcasts, mensajes directos, listados, consultas de información. Las operaciones de escritura (registro, desconexión, cambio de estado) son mucho menos frecuentes.

Un mutex normal (`pthread_mutex_t`) bloquea el acceso por completo. Si un hilo está leyendo la lista, ningún otro hilo puede leerla tampoco, aunque leer no modifica nada. Esto genera un cuello de botella innecesario: si 10 usuarios mandan un broadcast a la vez, los 10 hilos se forman en fila para acceder a la lista aunque ninguno la está modificando.

El **read-write lock** (`pthread_rwlock_t`) resuelve esto separando los accesos en dos tipos:

- **Read-lock** (`pthread_rwlock_rdlock`): cualquier número de hilos puede tomarlo al mismo tiempo. Mientras solo haya lectores, todos acceden a la lista en paralelo sin bloquearse.
- **Write-lock** (`pthread_rwlock_wrlock`): acceso exclusivo. El hilo que lo pide espera a que todos los lectores terminen, y una vez que lo tiene, nadie más puede leer ni escribir hasta que lo suelte.

Operaciones que toman read-lock (pueden ser simultáneas):

- Broadcast (iterar usuarios para enviar a todos).
- Mensaje directo (buscar al destinatario).
- Listado de usuarios.
- Consulta de información de un usuario.

Operaciones que toman write-lock (acceso exclusivo):

- Registro de usuario (insertar en la lista).
- Desconexión (eliminar de la lista).
- Cambio de estado (modificar un campo).
- Timeout de inactividad (modificar estado a INACTIVO).

6.2. Mutex por Socket

Además de la lista, existe otro recurso compartido: los sockets de los clientes. Cuando un hilo necesita enviar un mensaje a otro cliente (broadcast o mensaje directo), escribe al socket de ese cliente. Si dos hilos intentan escribir al mismo socket a la vez, los bytes se mezclan y el mensaje se corrompe.

Para esto se usa un **mutex** (`pthread_mutex_t`) por cada socket de cliente. Antes de escribir al socket de otro usuario, el hilo toma el mutex de ese socket, escribe, y lo suelta. Así, aunque varios hilos quieran enviar mensajes al mismo destinatario al mismo tiempo, los mensajes se entregan completos y uno a la vez.

Nótese que este es un mutex normal, no un rwlock, porque las operaciones sobre un socket individual son siempre de escritura — no hay caso en que dos hilos necesiten "leer" del mismo socket a la vez (cada hilo solo lee de su propio socket).

Referencias

- Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts* (10th ed.). Capítulo 3.
- `man 7 ip` — https://linux.die.net/man/7/ip